Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science, Semester V
Subject	CS3807 – Deep Learning Laboratory
Name	Madhav.S
Section	AIDS 'B'
Register Number	24011101066

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The goal of this experiment is to understand how a single artificial neuron works and to build a Single Layer Perceptron completely from scratch, without relying on any deep learning library. The experiment aims to build familiarity with the perceptron learning rule, demonstrate why activation functions matter, visualize the model's learning progress epoch by epoch, and evaluate its performance on a real-world dataset rather than a synthetic example.

2. Background Theory

An **artificial neuron** is the basic computing unit of a neural network. It takes in one or more input features, multiplies each of them by a weight, adds a bias term, and then passes the result through an activation function to produce an output.

The **Single Layer Perceptron** is the oldest and simplest of these models. It can only learn to separate two classes when they are (roughly) linearly separable, i.e., when a single straight line or flat plane can divide the data into its two classes.

Mathematical Formulation

Weighted Sum – every input is combined into a single number z:

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$ is the input vector (the feature values of one sample),
- $\mathbf{w}$ is the weight vector (how much importance each feature gets),
- b is the bias (shifts the decision boundary away from the origin),
- z is the resulting linear combination.

Prediction – the weighted sum is passed through an activation function f:

$$\hat{y} = f(z)$$

Weight Update Rule – after every wrong prediction, the weights are changed in the direction that would have made the prediction correct:

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update – the bias is updated the same way:

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

Here η (eta) is the **learning rate** – it controls how big a step is taken at each update.

3. Activation Functions

An activation function decides what a neuron actually outputs once it has computed its weighted sum. Without it, a neural network – no matter how many layers it has – would behave like a single linear equation, since stacking linear operations just gives another linear operation. Activation functions are what let networks learn curved, complex decision boundaries.

Step Activation Function

This experiment uses only the step function, since that is what the classical perceptron uses:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

It simply outputs 1 or 0 depending on which side of zero z falls on. Because of this hard cutoff, it can only ever draw a straight decision boundary, and it cannot be used for anything more advanced than basic linear classification.

Table 1: Common Activation Functions

Activation	**Output Range**	**Typically Used In**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary classification output layers
Tanh	$(-1, 1)$	Hidden layers of older-style networks
ReLU	$[0, \infty)$	Hidden layers of CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding the "dying ReLU" problem
Softmax	$(0, 1)$	Multi-class classification output layers

Note: only the Step function is used here. The rest will come up in later experiments once we move to Multilayer Perceptrons.

4. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository, fetched directly in code via the `ucimlrepo` package (`id=267`)
Link: `https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Table 2: Dataset Summary

Instances	1372
Features	4 numerical features
Classes	2
Missing Values	None
Task	Binary classification

The four numerical features that come from applying a wavelet transform to images of genuine and forged banknotes are:

- Variance
- Skewness
- Curtosis
- Entropy

Target Class:

- 0 – Authentic Banknote
- 1 – Forged Banknote

The classes are reasonably balanced: 762 authentic notes (55.5%) and 610 forged notes (44.5%), so accuracy alone is still a fairly trustworthy metric here.

5. Experimental Procedure

Task 1 – Dataset Exploration: The dataset was loaded using `fetch_ucirepo(id=267)`, and the features and target were combined into a single dataframe. Its shape, missing values, and descriptive statistics were examined before further processing.

Task 2 – Exploratory Data Analysis: Features histograms, a correlation heatmap, a pairwise scatter plot matrix, and boxplots were generated to understand the shape, spread, and relationships of the four features, along with a class-distribution plot to check class balance.

Task 3 – Data Preprocessing: All four features were standardized to zero mean and unit variance using `StandardScaler`, and the data was split into 80% training and 20% testing using stratified sampling so both sets preserve the same class ratio.

Task 4 – Perceptron Implementation: The perceptron was implemented entirely from scratch in NumPy – weight and bias initialization, the step activation function, forward propagation, and the perceptron learning rule -without using any machine learning library for this part.

Task 5 – Model Training: The perceptron was trained for 50 epochs with a learning rate of $\eta = 0.01$, logging the epoch number, number of misclassified samples, and the updated weights and bias after every epoch.

Task 6 – Model Evaluation: The trained model was evaluated on the test set using accuracy, precision, recall, F1-score, and a confusion matrix.

Task 7 – Learning Rate Study: Training was repeated from scratch with three different learning rates (0.001, 0.01, 0.1) to observe how the choice of η affects convergence behaviour.

Task 8 – Comparison with Scikit-learn (Optional): `sklearn.linear_model.Perceptron` was trained on the same train/test split, and its metrics were compared against the from-scratch implementation as a validation step.

In addition to the eight core tasks, five additional tasks were carried out: comparing the Step and Sigmoid activation functions, the Scikit-learn comparison above, the learning-rate study above, an experimental demonstration of why a single-layer perceptron cannot solve the XOR problem, and a study of how feature normalization affects convergence speed and stability.

6. Source Code

The complete implementation, exactly as run in Colab, is hosted on GitHub.

https://github.com/smadhav2024/Deep_Learning_Lab

The repository contains the full notebook deep learning lab 1.ipynb,the python file, the `requirements.txt` needed to run it, and this report's LaTeX source.

7. Experimental Results

Dataset Summary

Table 3: Descriptive Statistics of the Dataset

	Variance	Skewness	Curtosis	Entropy
mean	0.4337	1.9224	1.3976	-1.1917
std	2.8428	5.8690	4.3100	2.1010
min	-7.0421	-13.7731	-5.2861	-8.5482
25%	-1.7730	-1.7082	-1.5750	-2.4135
50%	0.4962	2.3197	0.6166	-0.5867
75%	2.8215	6.8146	3.1793	0.3948
max	6.8248	12.9516	17.9274	2.4495

No missing values were found in any column. After preprocessing, the split produced **1097 training samples** and **275 testing samples**.

Training Outcome

The perceptron was trained for 50 epochs at $\eta = 0.01$. Misclassifications fell sharply from 59 to about 20 within the first few epochs, then fluctuated between 12 and 24 for the remainder of training without reaching zero – expected behaviour, since the two classes overlap slightly and are not perfectly linearly separable. A sample of the training log:

```
Epoch  1 | Errors:  59 | Bias: -0.0300 | Weights: [-0.0680 -0.0938 -0.0678 -0.0110]
Epoch  5 | Errors:  20 | Bias: -0.0500 | Weights: [-0.0991 -0.1295 -0.1037  0.0102]
Epoch 10 | Errors:  20 | Bias: -0.0600 | Weights: [-0.1307 -0.1541 -0.1250 -0.0060]
Epoch 20 | Errors:  19 | Bias: -0.0700 | Weights: [-0.1636 -0.1946 -0.1613 -0.0064]
Epoch 30 | Errors:  17 | Bias: -0.0800 | Weights: [-0.1751 -0.2043 -0.1901 -0.0196]
Epoch 40 | Errors:  18 | Bias: -0.0900 | Weights: [-0.1948 -0.2244 -0.1853 -0.0197]
Epoch 50 | Errors:  18 | Bias: -0.1000 | Weights: [-0.1941 -0.2358 -0.2109 -0.0195]
```

The full epoch-wise log is summarized in Section 9, Performance Tables.

Test Set Evaluation

Table 4: Test Set Performance of the Scratch Perceptron (LR = 0.01, 50 epochs)

Metric	Value
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

Out of 275 test samples, 149 authentic notes and 122 forged notes were correctly classified. Only 4 authentic notes were wrongly flagged as forged, and **not a single forged note was missed** (0 false negatives). Failing to flag a forged note is a far costlier error than being being more cautious about a genuine note.

Learning Rate Study

Table 5: Effect of Learning Rate on Test Performance

Learning Rate	Accuracy	Epochs Run
0.001	0.9855	50
0.01	0.9855	50
0.1	0.9855	50

All three learning rates reached the same final test accuracy, and none converged to zero training error within 50 epochs. A higher learning rate (0.1) causes a much larger update in the weights and bias at every update, while a lower rate (0.001) updates very slowly.

Comparison with Scikit-learn

Table 6: Scratch Implementation vs. Scikit-learn's Perceptron

Model	Accuracy	Precision	Recall	F1-score
Scratch Perceptron	0.9855	0.9683	1.0000	0.9839
Scikit-learn Perceptron	0.9782	0.9677	0.9836	0.9756

Both implementations perform well. The scratch model slightly better on recall and F1-score, confirming that the manually implemented update rule was correct and consistent with the standard algorithm.

Decision Boundary (Optional)

Retraining using only two features (Variance and Skewness) for the sake of a 2D visualization resulted in a much higher training error – around 190 out of 1097 samples, roughly 17% – with little improvement over 50 epochs. This indicates that the remaining two features contribute meaningfully to separability in the full four-feature model.

Why a Single Layer Perceptron Cannot Solve XOR

XOR is not linearly separable: the points $(0, 0)$ and $(1, 1)$ belong to class 0, while $(0, 1)$ and $(1, 0)$ belong to class 1, and no single straight line can separate all the 0s from all the 1s. This was confirmed experimentally as the perceptron's error never dropped to zero on XOR, remaining stuck at 3–4 misclassifications out of 4 points for the entire run. Since a single-layer perceptron can only express a linear decision boundary, it cannot represent XOR – solving it requires at least one hidden layer (a Multilayer Perceptron).

Effect of Feature Normalization

Without normalization, a feature such as Skewness dominates the weighted sum and the resulting weight updates. The model trained on raw, unscaled features produced a much larger final bias ($+0.95$ versus -0.10) and noisier updates than the normalized version, confirming that standardizing features before training results in faster and more stable convergence.

8. Generated Plots with Interpretation

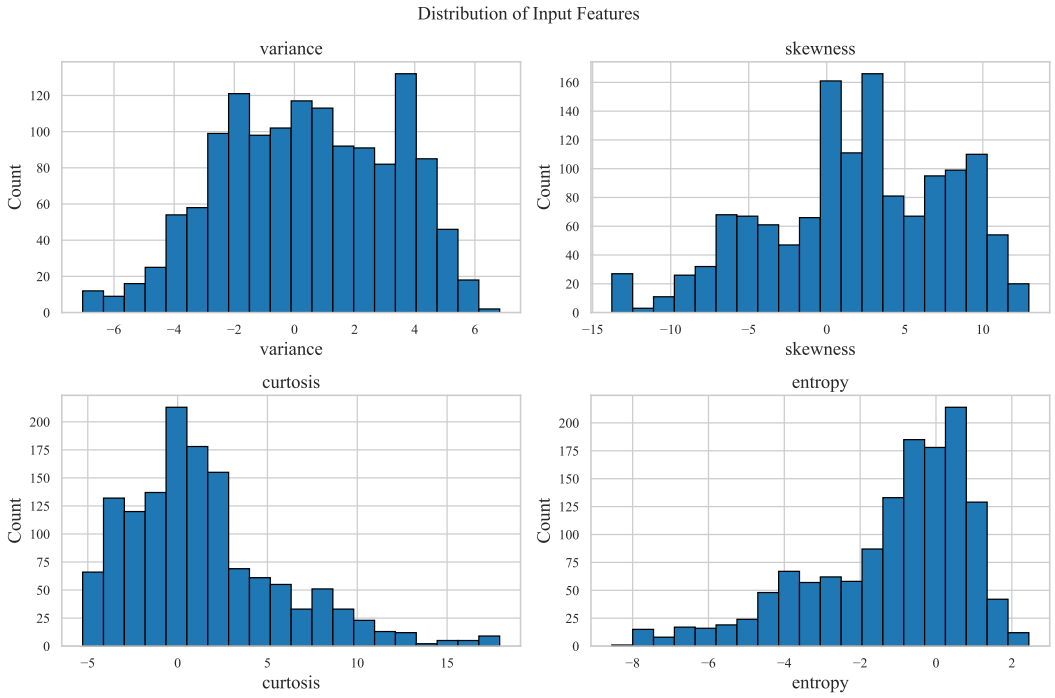

Figure 1: Feature Histograms

Observation: Variance and Entropy look fairly spread out and close to bell-shaped, while Skewness and Curtosis have a longer tail on the right with a few extreme values – meaning the raw features aren't on a comparable scale, which is exactly why standardizing them later matters.

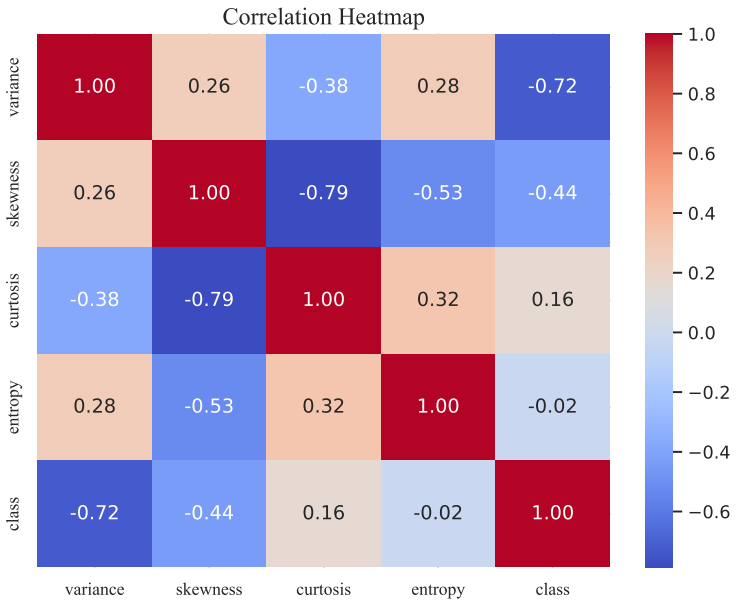

Figure 2: Correlation Heatmap

Observation: Variance has the strongest (negative) correlation with the class label, with Skewness close behind – these two features are doing most of the work for classification. Skewness and Curtosis are also correlated with each other, which makes sense since both describe the shape of the same underlying image distribution.

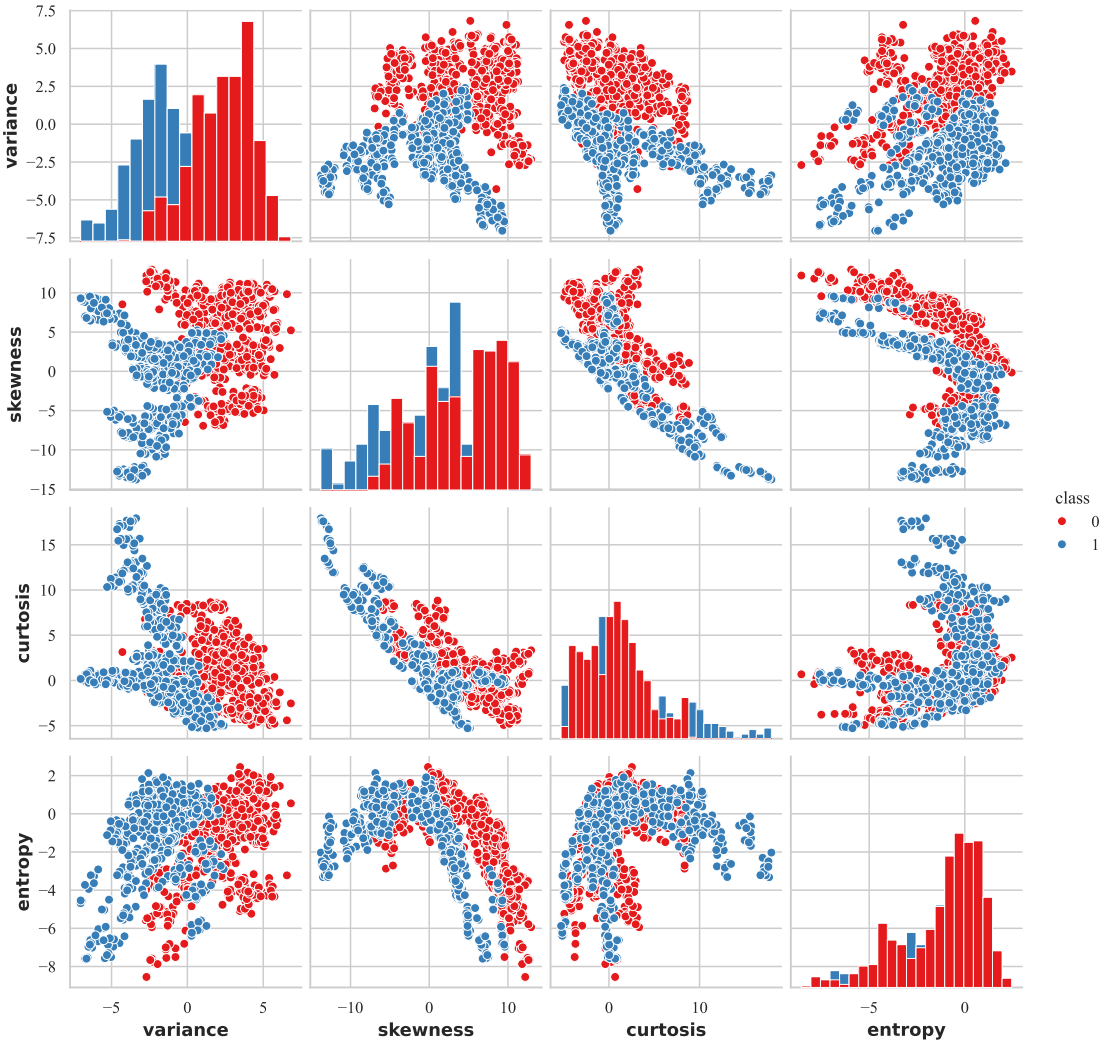

Figure 3: Pairwise Scatter Plot Matrix, Coloured by Class

Observation: Across every pair of features, the two classes form mostly separate clusters, especially along Variance and Skewness, with only a bit of overlap – meaning that a linear model like the perceptron will work well here.

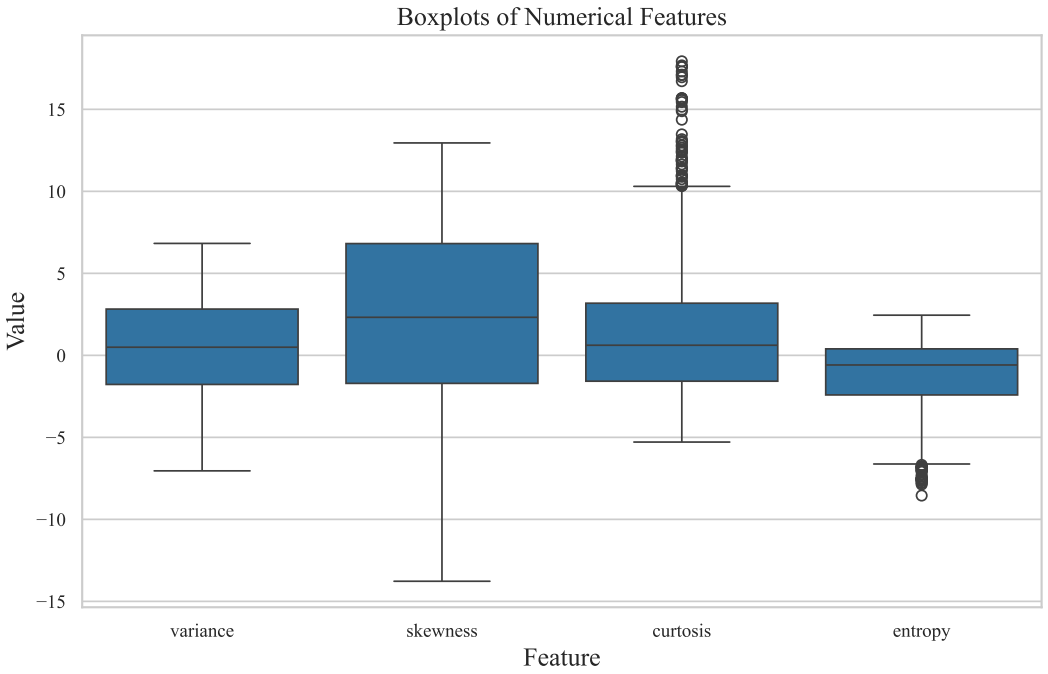

Figure 4: Boxplots of Features

Observation: Skewness and Curtosis have a fair number of outlier points beyond the whiskers, while Variance and Entropy are more tightly contained – another point explaining the importance of standardizing before training.

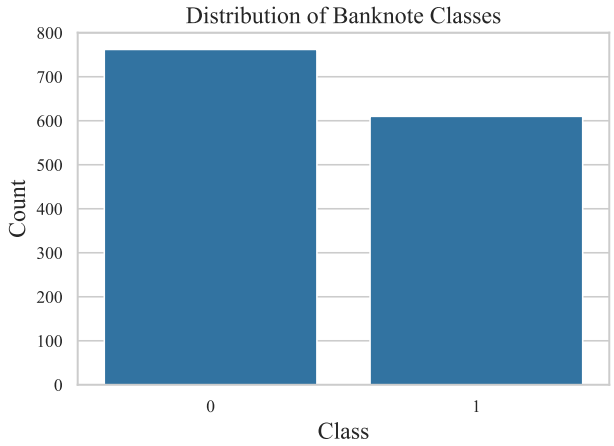

Figure 5: Class Distribution

Observation: The class split (55.5% vs. 44.5%) is close enough to balanced that no class-imbalance handling was needed for this experiment.

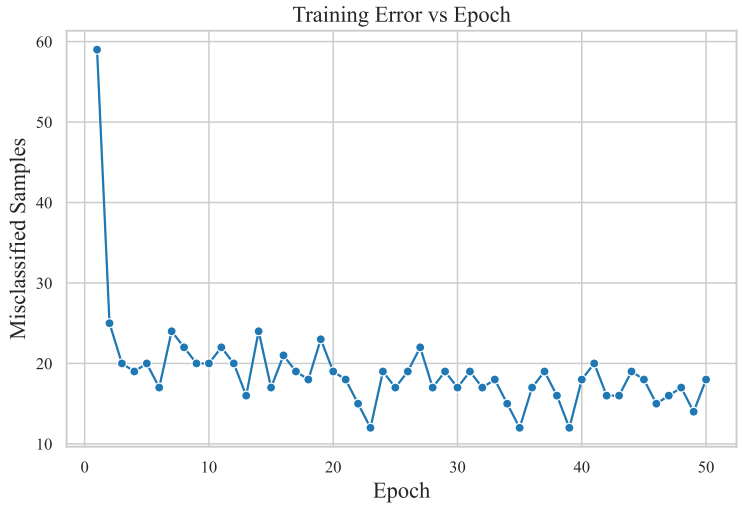

Figure 6: Training Error vs. Epoch

Observation: The error drops fast, from 59 down to about 20, within the first handful of epochs, and then hovers between roughly 12 and 24 for the rest of training, never quite reaching zero since the classes overlap slightly.

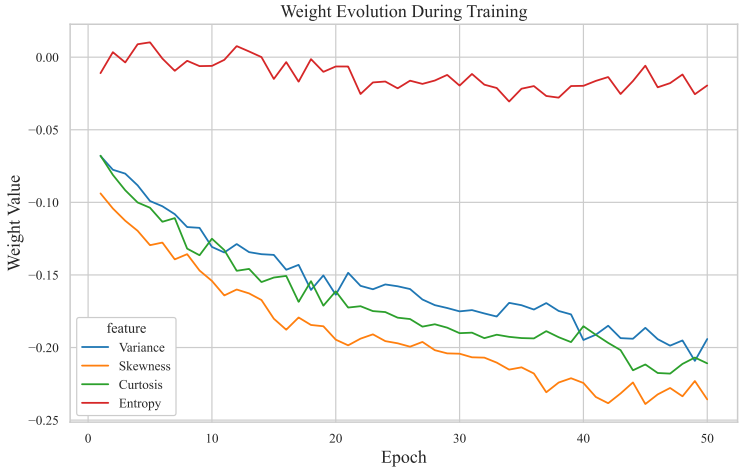

Figure 7: Weight Evolution over Epochs

Observation: All four weights move quickly away from zero early on and then settle. Variance, Skewness and Curtosis end up with the largest weights, while Entropy stays close to zero throughout, matching what the correlation heatmap already suggested.

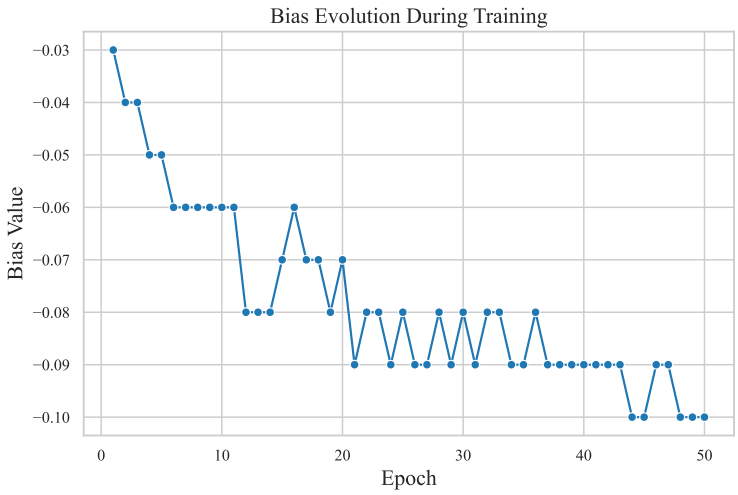

Figure 8: Bias Evolution over Epochs

Observation: The bias steadily drifts more negative and settles around -0.10, shifting the decision boundary away from the origin to better fit the data.

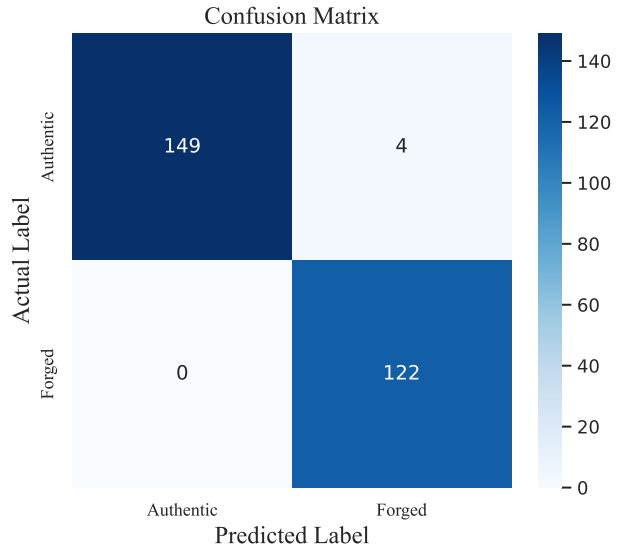

Figure 9: Confusion Matrix

Observation: 149 authentic and 122 forged notes were classified correctly, with only 4 false positives and zero false negatives – the model never let a forged note through undetected.

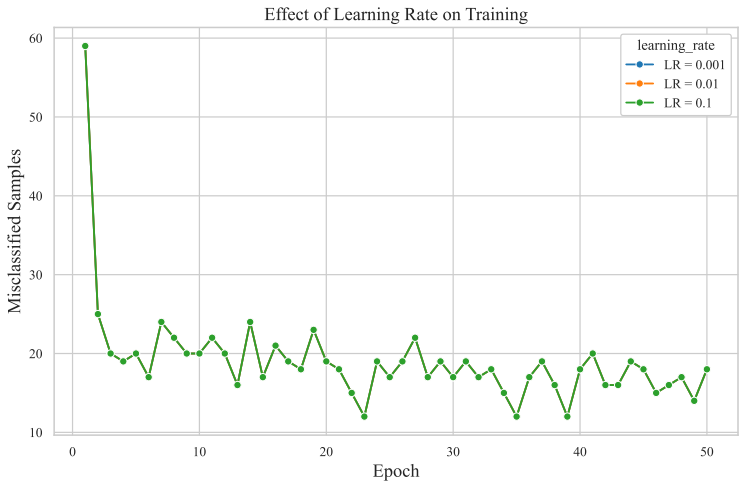

Figure 10: Learning Rate Comparison

Observation: All three learning rates reach the same final accuracy, but the internal error trajectory is visible more at the highest rate (0.1) and much smoother at the lowest (0.001).

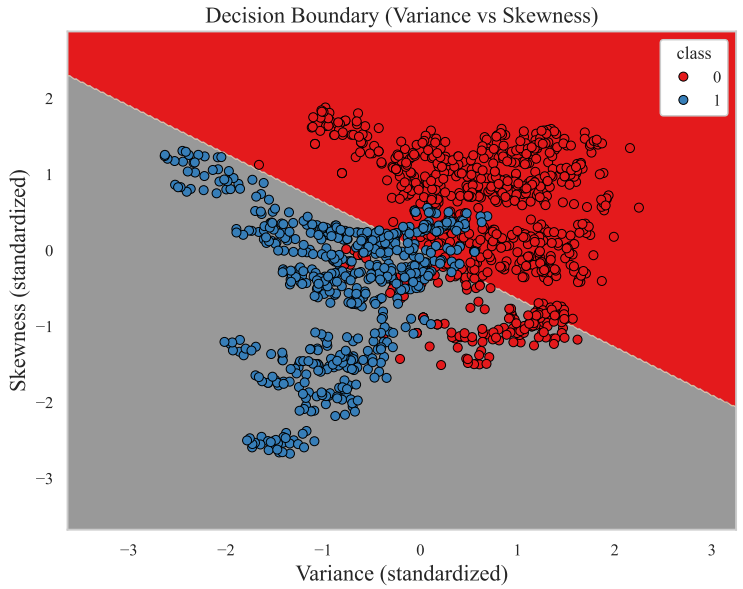

Figure 11: Decision Boundary using Variance and Skewness

Observation: The 2D boundary still separates most of the two classes reasonably well, but the visible overlap region is wider than it would be with all four features available.

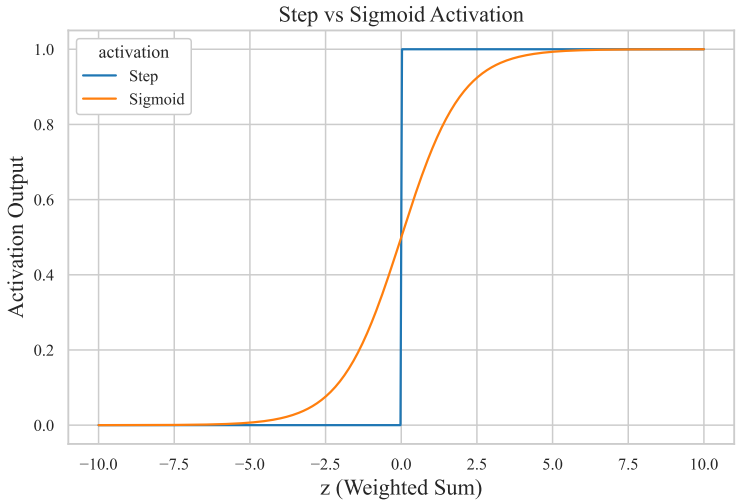

Figure 12: Step vs. Sigmoid Activation

Observation: The Step function jumps sharply between 0 and 1 with no usable slope anywhere, while the Sigmoid curve is smooth and differentiable across its entire range, which makes the gradient-based training possible.

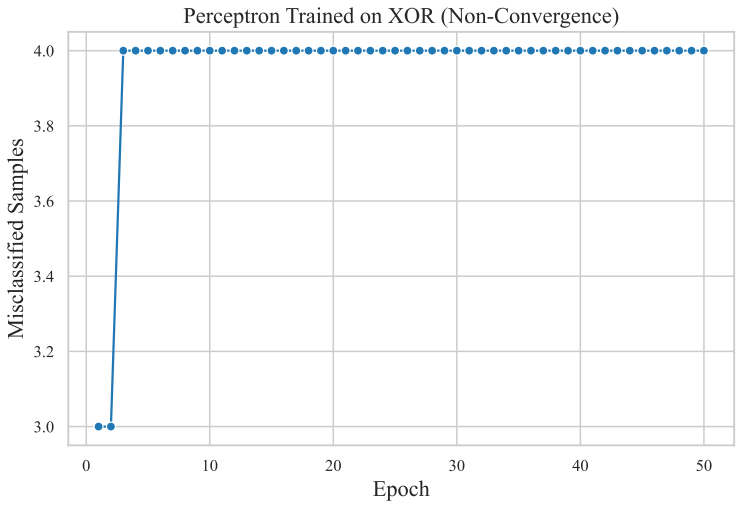

Figure 13: Perceptron Trained on XOR – Non-Convergence

Observation: The error never drops to zero and stays flat near 3–4 misclassifications out of 4 points for the entire run. This shows direct experimental proof that a single straight-line boundary cannot solve XOR.

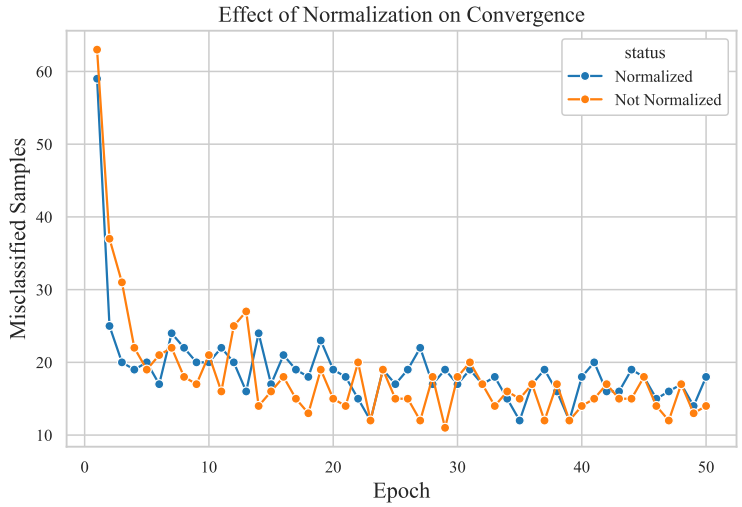

Figure 14: Effect of Normalization on Convergence

Observation: The normalized run's error curve is both lower overall and noticeably smoother than the raw-feature run's curve, making the benefit of standardization clearly visible.

9. Performance Tables

Table 7: Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275 (80% / 20%, stratified)
Learning Rate	0.01
Epochs	50
Final Weights	$[-0.1941, -0.2358, -0.2109, -0.0195]$
Final Bias	-0.1000
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

Table 8: Epoch-wise Learning (Selected Epochs)

Epoch	Errors	Weight 1 (Variance)	Weight 2 (Skewness)	Weight 3 (Curtosis)	Bias
1	59	-0.0680	-0.0938	-0.0678	-0.0300
5	20	-0.0991	-0.1295	-0.1037	-0.0500
10	20	-0.1307	-0.1541	-0.1250	-0.0600
20	19	-0.1636	-0.1946	-0.1613	-0.0700
30	17	-0.1751	-0.2043	-0.1901	-0.0800
40	18	-0.1948	-0.2244	-0.1853	-0.0900
50	18	-0.1941	-0.2358	-0.2109	-0.1000

10. Discussion

This experiment provides a clear, practical demonstration of how a perceptron learns, beyond the theoretical formulation alone. The model reached 98.55% accuracy with perfect recall on the test set, consistent with the EDA pairplot, which suggested the dataset is close to linearly separable – explaining why a simple linear model such as the perceptron performs well on it.

Several noticeale observations were :

- The learning rate had little effect on *final* accuracy in this case, but it clearly changed *how* the model reached that point – larger learning rates cause much larger, more erratic swings in the weights and bias during training.
- Cross-checking against Scikit-learn's own Perceptron served as a useful validation step, confirming that the from-scratch implementation was identical to the other perceptron.
- Reducing the feature set to two dimensions for the decision-boundary plot noticeably degraded performance, indicating that even features with small final weights (such as Entropy) contribute meaningfully in combination with the others.
- The XOR experiment made the linear-separability limitation directly observable rather than purely theoretical – the error remained permanently stuck. It showed that perceptron can only show linear decision boundaries.
- The Step activation function is the central limitation of this approach: it can only express straight-line decision boundaries, provides no confidence or probability estimate,sometimes not differentiable etc. This is precisely why it is replaced by smooth, differentiable activations such as Sigmoid and ReLU in deeper, multilayer networks.

11. Conclusion

- Built a Single Layer Perceptron completely from scratch in NumPy and trained it on the real-world Banknote Authentication dataset.
- Achieved **98.55% accuracy**, **96.83% precision**, **100% recall**, and a **98.39% F1-score** on the held-out test set.
- Validated the implementation against Scikit-learn's Perceptron, which scored a close **97.82% accuracy** – confirming the from-scratch model was implemented correctly.
- Learning rate (0.001 / 0.01 / 0.1) changed the training dynamics but not the final accuracy on this dataset.
- Feature normalization made a real, visible difference to how smoothly and reliably the model converged.
- Demonstrated experimentally that a single-layer perceptron cannot solve the XOR problem, since XOR is not linearly separable – this is the exact reason Multilayer Perceptrons exist.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.